

# MediReservas

Plataforma de Reservas para Consultas Medicas  
Arquitectura de Microservicios

| Elemento  | Detalle                                             |
|-----------|-----------------------------------------------------|
| Version   | MVP 1.0                                             |
| Stack     | Node.js, Express, PostgreSQL, NGINX, Docker Compose |
| Servicios | Auth, Agenda, Notifications, Records, API Gateway   |
| Ejecucion | <code>docker compose up --build</code>              |

## Descripcion del problema

Las clinicas pequenas suelen coordinar citas por telefono o mensajeria, lo que provoca doble reserva de horarios, baja trazabilidad, dificultad para administrar disponibilidad medica y poca visibilidad para el paciente. MediReservas centraliza el proceso sin convertir todo el sistema en un monolito.

## Alcance del MVP

El MVP permite registrar usuarios, iniciar sesion, listar medicos, publicar disponibilidad, agendar citas, cancelar citas, emitir notificaciones internas y registrar resultados basicos de consulta. No incluye pagos, videollamadas, recetas certificadas ni integraciones hospitalarias externas.

## Requerimientos funcionales

- Registro e inicio de sesion de usuarios.
- Roles de paciente, medico y administrador.
- Registro o actualizacion de perfil profesional del medico.
- Publicacion de bloques de disponibilidad.
- Consulta de medicos y horarios disponibles.
- Agendamiento y cancelacion de citas.
- Generacion de notificaciones internas.
- Registro de resultados medicos por el medico.
- Consulta de resultados medicos por el paciente.

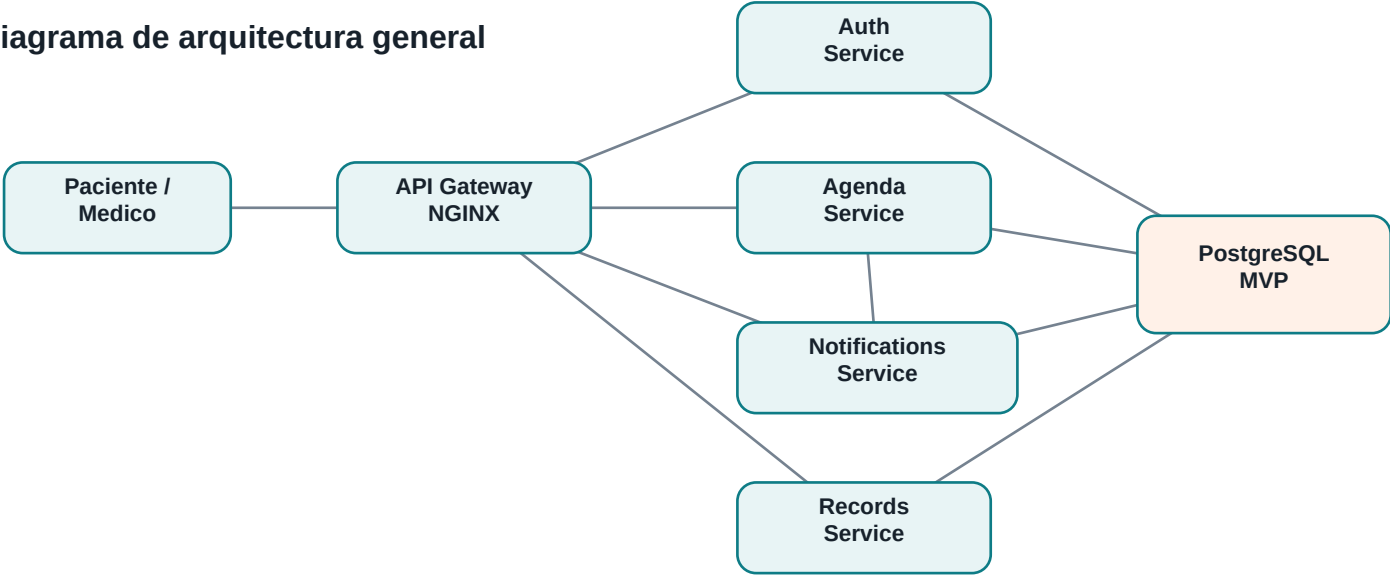
## Requerimientos no funcionales

- Modularidad por dominio y contratos HTTP.
- Ejecucion local con Docker Compose.
- Seguridad mediante token firmado.
- Trazabilidad con estados y fechas de creacion.
- Consistencia de agenda mediante transaccion SQL y bloqueo de horario.
- Mantenibilidad por separacion de responsabilidades.

# Arquitectura y diagramas

NGINX funciona como API Gateway y punto de entrada unico. Los servicios internos exponen APIs HTTP. Agenda valida tokens contra Auth y emite un evento HTTP interno hacia Notifications cuando una cita se agenda o cancela.

Diagrama de arquitectura general



| Actor         | Casos de uso                                                                                                           |
|---------------|------------------------------------------------------------------------------------------------------------------------|
| Paciente      | Registrarse, iniciar sesion, consultar medicos, agendar cita, cancelar cita, ver notificaciones, consultar resultados. |
| Medico        | Iniciar sesion, registrar perfil, publicar disponibilidad, consultar agenda, registrar resultados.                     |
| Administrador | Gestionar datos base y apoyar tareas operativas del MVP.                                                               |

| Secuencia para agendar cita | Interaccion                                                                     |
|-----------------------------|---------------------------------------------------------------------------------|
| 1                           | Paciente envia POST /api/agenda/appointments al API Gateway.                    |
| 2                           | Agenda Service valida el token con Auth Service.                                |
| 3                           | Agenda Service bloquea el horario disponible y crea la cita en PostgreSQL.      |
| 4                           | Agenda Service marca el horario como booked.                                    |
| 5                           | Agenda Service solicita a Notifications Service crear una notificacion interna. |
| 6                           | El paciente recibe confirmacion de cita creada.                                 |

# Stack tecnologico

| Capa            | Tecnologia        | Justificacion                                                          |
|-----------------|-------------------|------------------------------------------------------------------------|
| Gateway         | NGINX             | Entrada unificada, reverse proxy y ocultamiento de servicios internos. |
| Servicios       | Node.js + Express | Ligero, rapido para APIs REST y conocido para microservicios.          |
| Persistencia    | PostgreSQL 16     | Transacciones ACID para evitar doble reserva.                          |
| Frontend        | HTML/CSS/JS       | Interfaz simple para demostrar el flujo del MVP.                       |
| Infraestructura | Docker Compose    | Ejecucion local reproducible multi-contenedor.                         |

# Modelo de datos

| Entidad                   | Responsabilidad                 |
|---------------------------|---------------------------------|
| auth_users                | Usuarios, credenciales y roles. |
| agenda_doctors            | Perfil profesional del medico.  |
| agenda_availability_slots | Bloques de disponibilidad.      |
| agenda_appointments       | Citas entre paciente y medico.  |
| notification_messages     | Notificaciones internas.        |
| record_results            | Resultados o notas de consulta. |

# Contratos de API

El contrato OpenAPI completo esta en docs/openapi.yml.

| Dominio       | Endpoints principales                                                                 |
|---------------|---------------------------------------------------------------------------------------|
| Auth          | POST /api/auth/register, POST /api/auth/login, GET /api/auth/me                       |
| Agenda        | GET /api/agenda/doctors, POST /api/agenda/availability, POST /api/agenda/appointments |
| Notifications | GET /api/notifications/me, PATCH /api/notifications/{id}/read                         |
| Records       | POST /api/records/results, GET /api/records/me                                        |

## Decisiones tecnicas y trade-offs

- Se usa NGINX como gateway para mantener un punto de entrada claro y desacoplar clientes de servicios internos.
- La comunicacion entre servicios es HTTP sincrona por simplicidad del MVP; en produccion podria usarse RabbitMQ o Kafka para eventos.
- Se usa una sola instancia PostgreSQL local con tablas por dominio para simplificar la demostracion; en produccion cada servicio tendria almacenamiento aislado.
- Auth Service centraliza validacion de tokens para no duplicar logica de seguridad.
- Agenda Service usa transacciones y bloqueo de fila para evitar doble reserva del mismo horario.

## Evidencia del sistema

- Frontend disponible en <http://localhost:8081>.
- Login probado con [luis.paciente@demo.com](mailto:luis.paciente@demo.com) y [ana.doctor@demo.com](mailto:ana.doctor@demo.com).
- Prueba realizada: medico publica disponibilidad, paciente agenda cita y se genera notificacion.
- Prueba adicional: medico registra resultado y paciente consulta historial.
- Smoke test automatizado disponible en [scripts/smoke-test.ps1](#).
- Contenedores verificados: postgres, auth, agenda, notifications, records y gateway.

## Instrucciones de ejecucion

| Paso | Comando o accion                                                |
|------|-----------------------------------------------------------------|
| 1    | cd "fernando proyecto analisis"                                 |
| 2    | cp .env.example .env                                            |
| 3    | docker compose up --build                                       |
| 4    | Abrir <a href="http://localhost:8081">http://localhost:8081</a> |

## Conclusiones

MediReservas evidencia separacion de responsabilidades, contratos de API, comunicacion entre servicios, despliegue local con contenedores y una solucion ejecutable coherente con microservicios.